Laboratory Report — Logger and Parentheses Validator

Student: Andrei Varvariuc

Programming language: Java

1. Program Structure

`Logger.java` file contains the `Logger` class. It is responsible for handling log messages defined by the programmer. It allows the developer to print certain messages with a specified log level. Each logger has access to a static variable called `level` which controls whether the log with a certain log level should be printed or not.

`ParenthesesValidator.java` file contains the `ParenthesesValidator` class. It provides a function to test whether a string consisting of parentheses is valid or not. In the test function it utilizes a `Logger` to log certain actions.

2. Classes and Objects

A class is a special object that can store attributes and methods. An attribute is a variable tied to the class. A method is a function tied to the class.

Here is an example of an object creates from a class:

```
    // Main.java, line 4
    Logger loggerTest = new Logger("test");
```

An example of an object created from a class, or in other words an instance of a class, can be found in `Main.java` on line 4.

Since the only non-static attribute is the `name` attribute, it is the only attribute each instance stores. All other attributes are stored in one place and not in the instance itself to optimize space.

3. Static and Instance Members

Usually, class attributes are not shared across class instances, usually called instance attribute, but with `static` keyword an attribute is shared across all instances. Let's take `Logger` as an example. Its attributes include `level`, `format`, `name`, and a few others. The `level` and `format` attributes are static, meaning they are shared between instances, but `name` is an instance attribute. If we have two instances, the `name` attribute of those instances will be different.

The `level` attribute is static to control every single logger at once. Same for `format` attribute: we want to change it in one place and see the change reflected in all loggers at once.

Since the static attributes are shared across all instances, that means that, for example, changing the `level` attribute changes it for all instances, but changing the `name` attribute will only change it for that specific instance.

```
public class Logger {
  public String name;  // instance attribute - stored in the instance
  public static Level level = Level.INFO;  // static attribute - shared across
instances
  public static String format = "[%t] [%l] %n (%r): %m"; // static attribute

  // methods
  // ...
  // ...
}
```

4. Methods and Responsibilities

Unlike with attributes, the `static` keyword for methods only signals that that method does not use non-static attributes. Methods include `log`, `setLogLevel`, `setFormat`, and others. `setLogLevel` and `setFormat` are both static methods because they operate with static attributes only. `log` method on the other hand needs to know the `name` attribute, which is tied to the instance, so it is not static.

The `log` method is responsible for taking the log level, message string and print it to the console if the specified log level is not lower than the lower boundary `level` in each `Logger`.

```
public class Logger {
  // ...
  // ...
  // attributes

  public static void setFormat(String format) { ... }  // static method

  public static void setLogLevel(Level newLevel) { ... }  // static method

  public void log(Level logLevel, String message) { ... }  // instance method
}
```

`ParenthesesValidator` uses `Logger` class instead of implementing logging itself because that is what classes are for - reusing similar code.

Here is an example of a method call:

```
// ParenthesesValidator.java, line 10
logger.log(Logger.Level.TRACE, "opening parenthesis found");
```

5. Testing and Correctness

Valid input: `"()"`
Invalid input: `"))"`

To determine if the parentheses are valid using an interesting trick. Start at number zero and start reading the string from left to right. When you see an opening parenthesis, add 1 to the counter, and when you see a closing parenthesis subtract 1. If that number ever goes below 0, that means there is an extra closing parenthesis, meaning it is invalid. Only when that condition is false is the string valid. An edge case is if there is a non-parenthesis symbol, in which case the program simply considers the string invalid.

There is an issue though, and a simple test case to show that is `"("`. The function should return false, but it returns true. That is because we forgot to account for the fact, that when we reach the end of the string, all parentheses should have been paired up, resulting in the final number being 0, but in the case above the final number is 1. So in the end we also have to check if the number is 0, and only when it is the string is valid.

6. Reflection

The most important OOP concept applied in this laboratory work are classes.

I think the program is already complete as it is. If I were given more time I'd probably spend time only checking for bugs, typos, or other small errors.